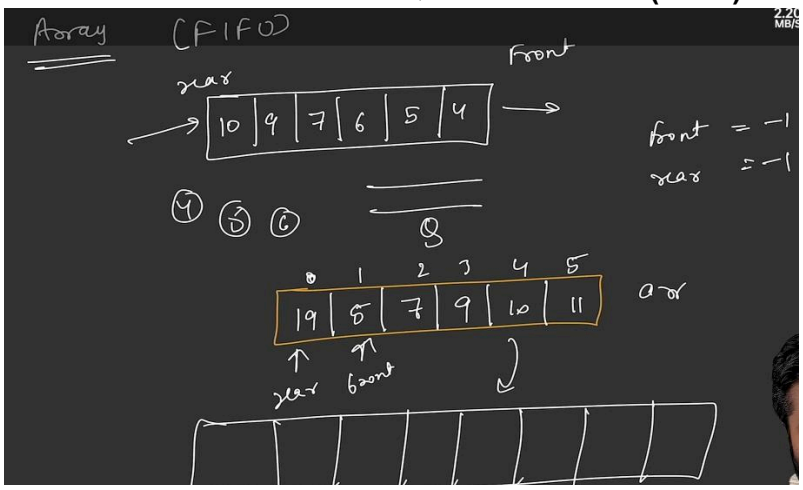
Queue Interface

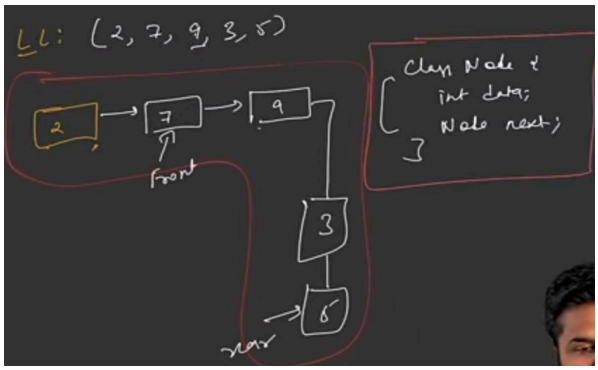
To use **Queue Interface** & **Stack Interface**, we have **2 ways—->**

- **ArrayList.**
- **LinkedList.**

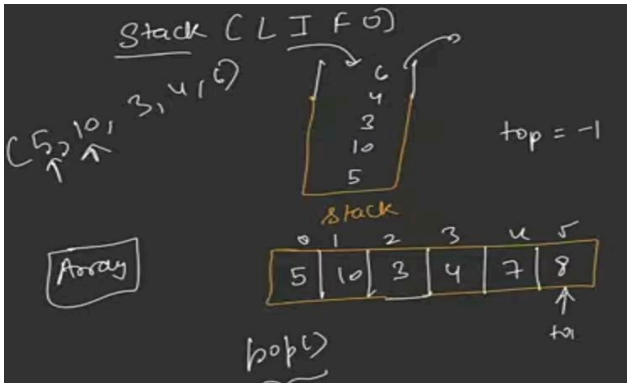
Queue Interface (FIFO) through ArrayList Path**—>**



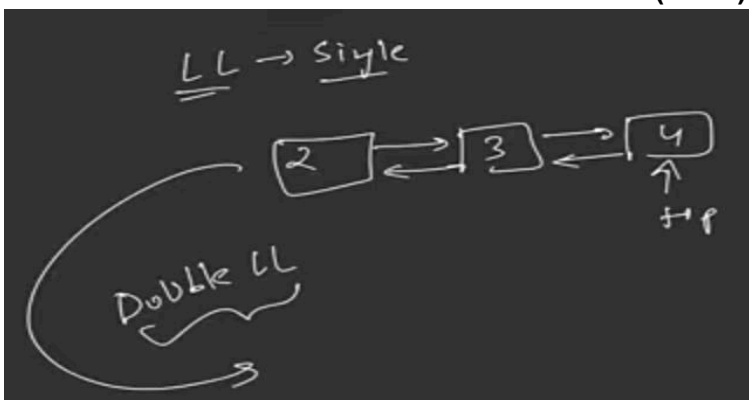
Queue Interface (FIFO) through LinkedList Path→

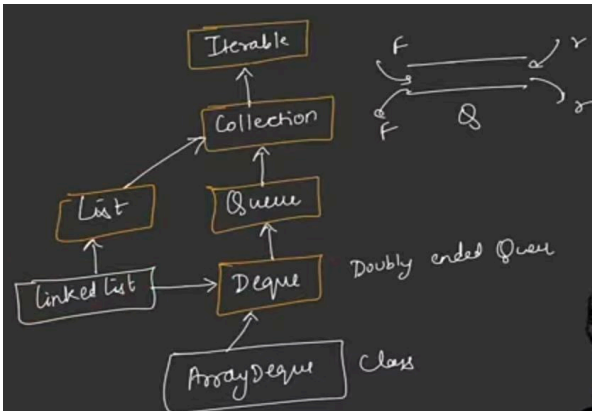


Stack Interface (LIFO) through ArrayList Path.



Stack Interface (LIFO) through LinkedList Path.





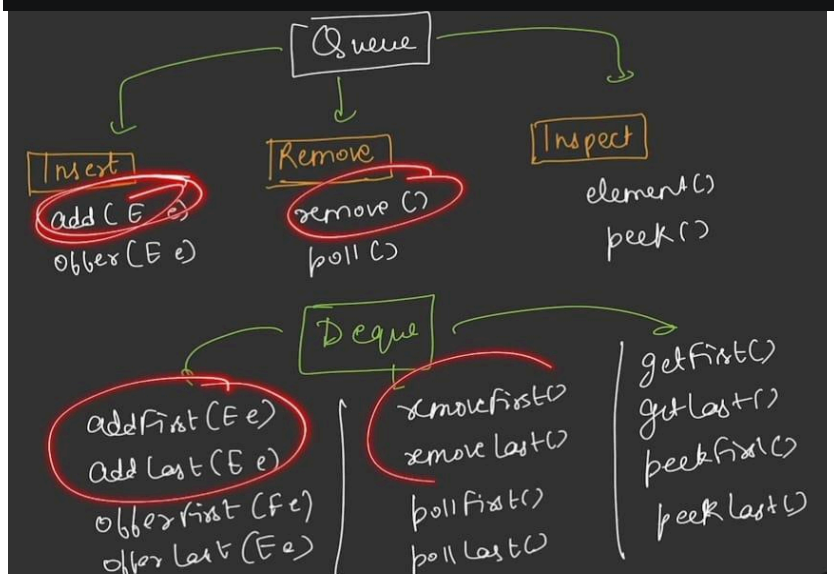
- ArrayDqueue class or linkedList Class hmare queue interface or stack interface dono ko implement krti ha.
- Kuki ye doubley ended queue ko implement krti ha.

Which class among both **(ArrayDqueue Class & LinkedList Class)** is better for using **QUEUE INTERFACE.**
Answer is—> ArrayDqueue.
Because, Array is cash friendly, kuki ye apne sath wale indexes ka cash ko bhi sath leke chalta ha.
Or hme Queue & stack me pointers ke sath kaam krna pdta ha.
Facts of ArrayDqueue→

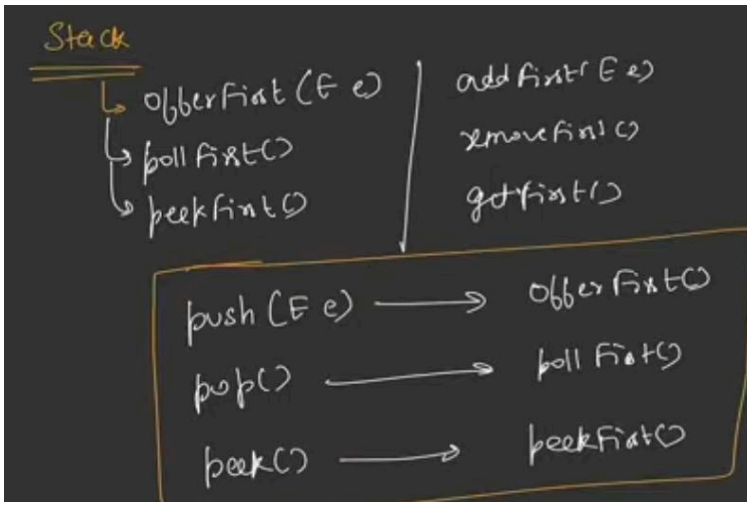
- We can't insert null in ArrayDqueue, whereas we can insert null in linkedList, therefore ArrayDqueue is more stricter then linkedList.
- And in Queue and Stack we prefer not to use null as an element.

Methods of queue & Deque interfaces→

```
ArrayDeque<Integer> queue = new ArrayDeque<>();
    //Single ended queue methods -->
        // Addition methods / enqueue methods.
            queue.add(1);// it throws exception if it fails.
            queue.offer(2);// it will never throw excption when it fails but shows false..
            // therfore offer is more safer then add method.
            queue.offer(3);
        // front access methods.
            System.out.println(queue.peek());// only to se the front/first element in queue.
            System.out.println(queue.element());// same as peek but throw exception if there is no element in
queue.
        // remove element methods.
            queue.remove();// unsafe--> throws exception.
            queue.poll(); // safer--> return null.
```



- All methods can be call in ArrayDeque & linkedList also.
- And for STACK i can use Deque's (Addfirst(); removeFirst(); getFirst();)

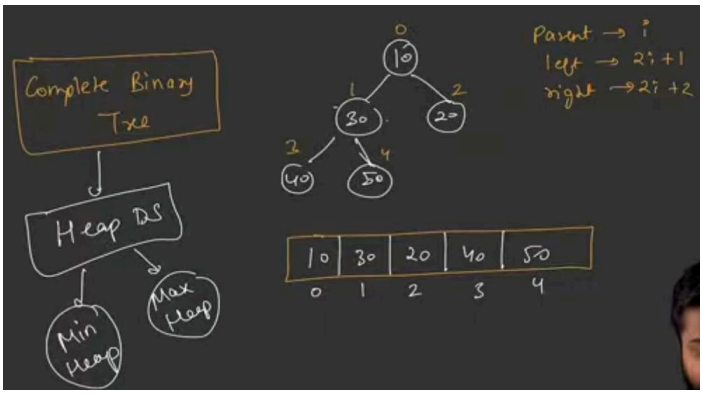


(Priority Queue class) directly implements the QUEUE interface.

- It is a special kind of queue,
- Here elements does'nt follow the FIFO property.
- On applying the method pq.poll front element remove nahi hoga blki highest priority wala remove hoga.
- Priority aksar defaultly sabse chote element ko milte ha.

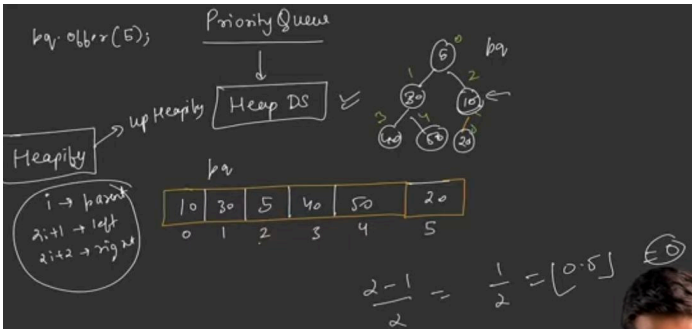
How its priority works→ Internal implementation of priority queue.

- Internally it implements a new dataStructure called **HEAP DATASTRUCTURE.**
- It uses Binary Heap.

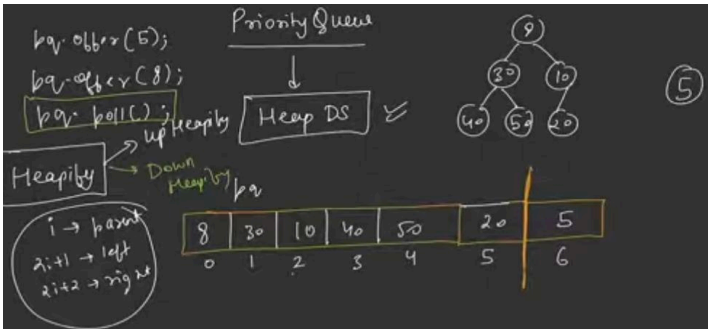


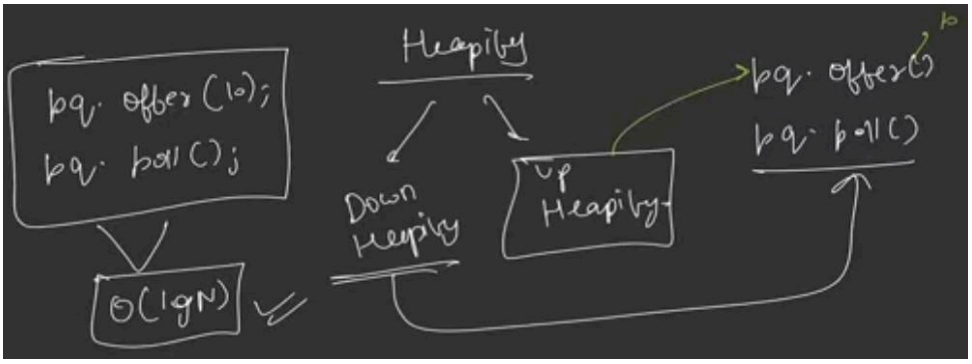
Heapify –:>

upHeapify:--> new value ko top ke liye swap krna.



DownHeapify→kisi bhi value ko pollout krne k liye, value ko last me lana or fir values ko swap krna.






```java
// Min-Heap
      PriorityQueue<Integer> pq = new PriorityQueue<>();
          pq.offer(10 );
          pq.offer(30 );
          pq.offer(20 );
          pq.offer(40 );
          pq.offer(50 );
              System.out.println(pq.poll());
              System.out.println(pq.poll());
              System.out.println(pq.poll());
              System.out.println(pq.poll());
              System.out.println(pq.poll());
              System.out.println(pq.poll());
      //Max-Heap
      PriorityQueue<Integer> pq2 = new PriorityQueue<>((a,b)-> b-a);
          pq2.offer(10 );
          pq2.offer(30 );
          pq2.offer(20 );
          pq2.offer(40 );
          pq2.offer(50 );
              System.out.println(pq2.poll());
              System.out.println(pq2.poll());
              System.out.println(pq2.poll());
              System.out.println(pq2.poll());
              System.out.println(pq2.poll());
              System.out.println(pq2.poll());
```

Methods of PRIORITYQUEUE

